



NAAN MUDHALVAN ARTS INTERNSHIP PROGRAM 2026

Frontend Web Development using React

Project Submission Template

STUDENT NAME: KRISHNA M

YEAR: 2024-2027

COLLEGE CODE:BRUAA

COLLEGE NAME: SAN INTERNATIONAL COLLEGE OF ARTS AND SCIENCE

DEPARTMENT:COMPUTER SCIENCE

SEMESTER: 4

ROLL NO: 2422J0075

NMID: 92AA22D2E25D7E99DE14991C003CD3C4

DATE: 17-06-2026

TITLE: API based covid health tracker

GITHUB LINK : <https://github.com/krxnabh>

1. ABSTRACT

The API-Based COVID/Health Stats Tracker is a web application developed using React that provides real-time COVID-19 and health-related statistics from a public API. The application helps health awareness organizations monitor and display important health information for different countries. Users can search for a country and view live statistics such as total cases, active cases, recovered cases, deaths, and recovery rates. The system also provides conditional alerts for high-risk countries based on predefined thresholds. The project demonstrates the use of React components, API integration, state management, and dynamic data rendering to create an informative and user-friendly dashboard.

2. PROBLEM STATEMENT

During health emergencies such as COVID-19 outbreaks, accessing accurate and real-time information is essential. Many users find it difficult to obtain updated statistics from multiple sources. Health awareness organizations require a centralized platform that can display reliable and current health data to educate the public and support awareness campaigns. Therefore, a web application is needed to fetch and display live health statistics in an easy-to-understand format.

3.OBJECTIVE

- To develop a React-based web application for displaying live COVID-19 statistics.
 - To integrate public APIs for real-time data retrieval.
 - To implement country-wise search functionality.
 - To display important statistics such as total cases, recovered cases, active cases, and deaths.
 - To generate alerts for high-risk countries based on case counts.
 - To provide an interactive and responsive user interface.
-

4. PROJECT SCOPE

Features Included

- Fetch live COVID-19 data from a public API.
- Display worldwide and country-specific statistics.
- Search countries by name.

Show key metrics including:

- Total Cases
- Active Cases
- Recovered Cases
- Deaths
- Critical Cases
- Conditional alerts for high-risk countries.
- Responsive design for desktop and mobile devices.

Future Enhancements

- Vaccination statistics tracking.
- Graphical data visualization using charts.
- Historical trend analysis.
- User notifications and alerts.
- Multi-language support.

Limitations

- Data accuracy depends on the external API source.
 - Internet connection is required for fetching live data.
-

5. TECHNOLOGIES USED

Technology	Purpose
React.js	Frontend Framework
JavaScript (ES6+)	Application Logic
HTML5	Structure
CSS3	Styling
Axios / Fetch API	API Requests
React Hooks	State Management
Public COVID API	Live Data Source
VS Code	Development Environment
Git & GitHub	Version Control

6. APPLICATION FLOW

Step 1: Application Launch

The user opens the COVID/Health Stats Tracker in a web browser.

Step 2: Data Fetching

The application sends an API request to the COVID statistics API and retrieves the latest data.

Step 3: Data Display

Global health statistics are displayed on the dashboard.

Step 4: Country Search

The user enters a country name in the search bar.

Step 5: Filtering

The application filters and displays data related to the selected country.

Step 6: Alert Generation

The system checks whether the country's active cases exceed a predefined threshold.

Step 7: Conditional Alert

If the country is classified as high-risk, a warning message is displayed.

Step 8: Updated Information

The dashboard continuously reflects the latest data whenever the user performs a search or refreshes the page.

7. KEY REACT CONCEPTS USED

Components

The application is divided into reusable components such as:

- Header Component
- Search Component
- Statistics Card Component
- Alert Component

JSX

Used for creating dynamic user interface elements.

useState Hook

Manages:

- Search input
- API data
- Alert status

useEffect Hook

Fetches API data when the application loads.

Props

Used to pass data between parent and child components.

Conditional Rendering

Displays warning alerts only for high-risk countries.

Event Handling

Handles user interactions such as searching for countries.

API Integration

Retrieves real-time data using Axios or Fetch API.

8. EXPECTED OUTCOME

Home Dashboard

Displays:

Total Cases

Active Cases

Recovered Cases

Deaths

Critical Cases

Search Functionality

Users can search for any country and instantly view its statistics.

High-Risk Alert Example

⚠ Warning!

This country is categorized as High Risk.

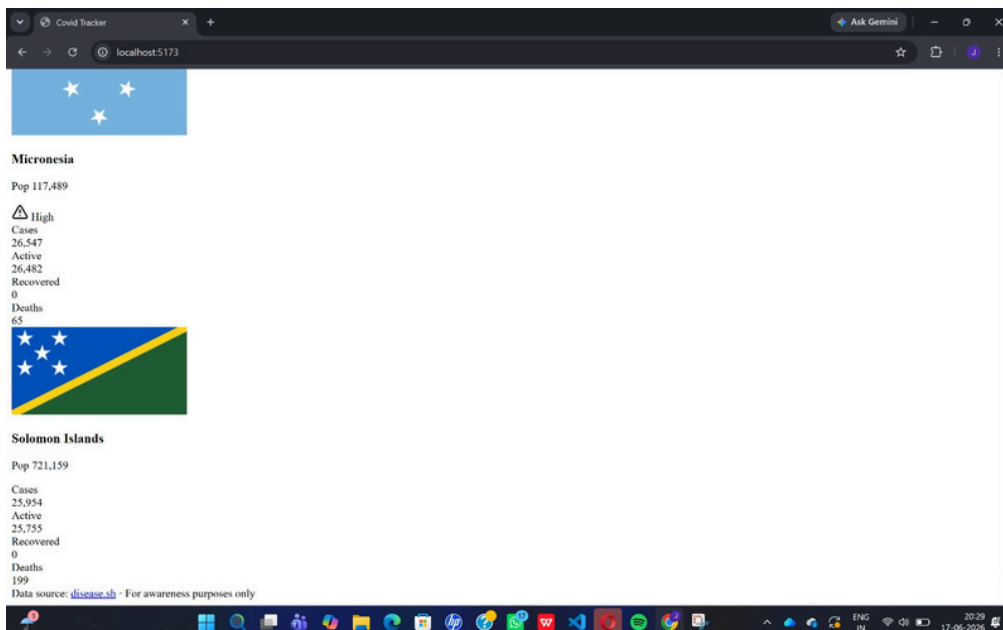
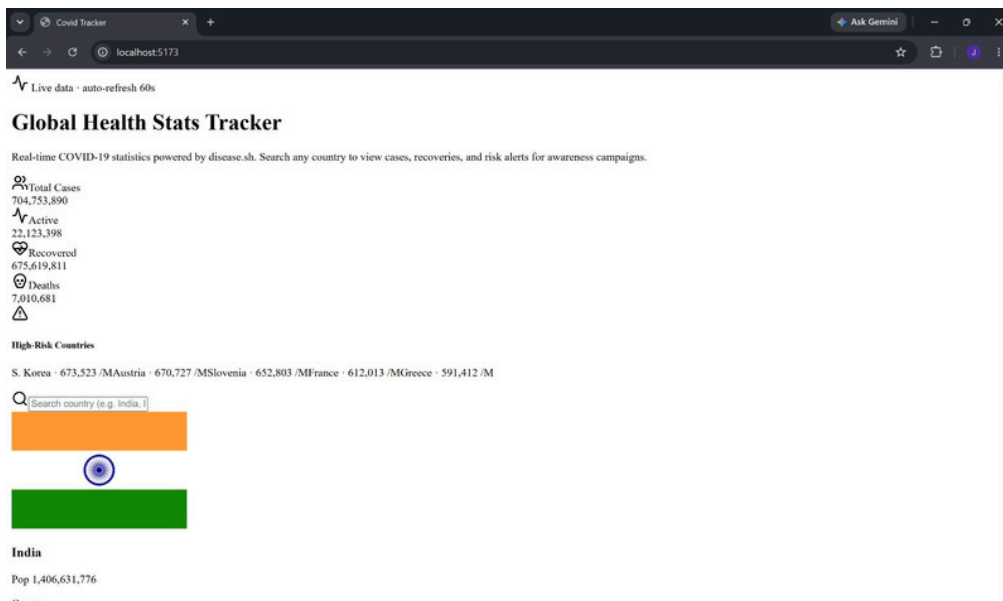
Active Cases: 1,000,000+

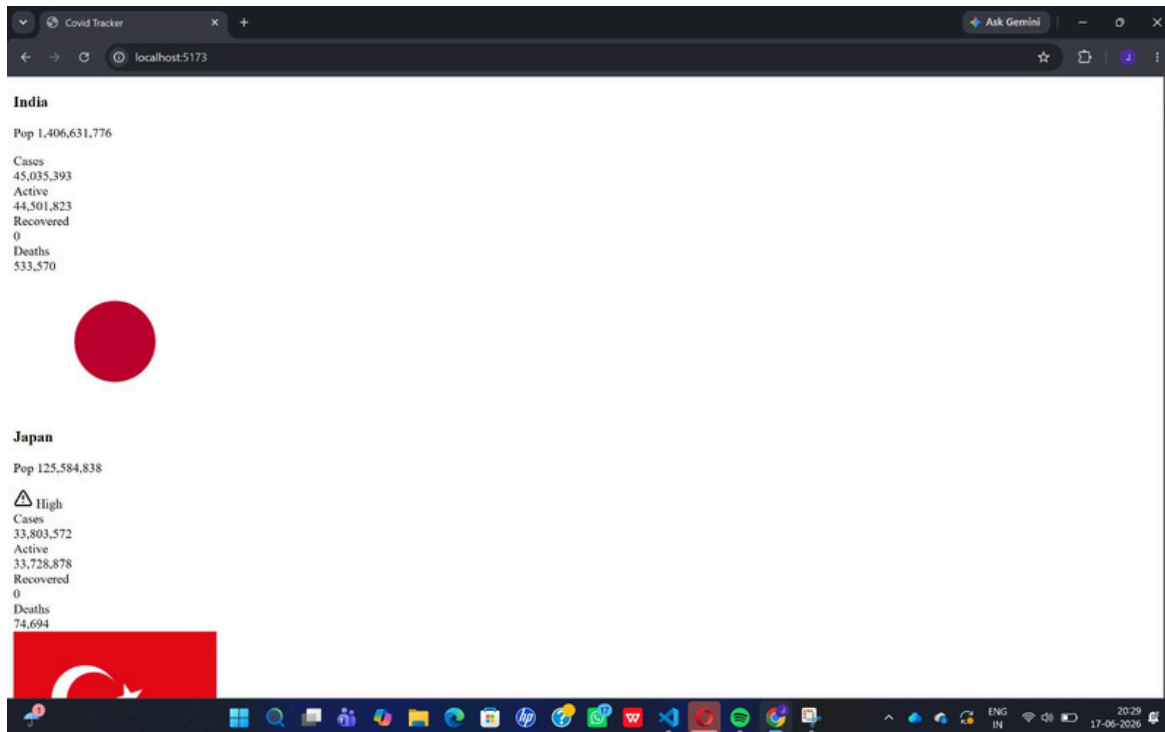
Please follow health guidelines.

Responsive Interface

The application works efficiently on:

- Desktop
- Tablet
- Mobile Devices





9. CHALLENGES FACED

API Data Handling

Challenge: Understanding API response structures.

Solution: Used browser developer tools and console logging to inspect API data.

State Management

Challenge: Updating UI dynamically after data retrieval.

Solution: Implemented React Hooks such as useState and useEffect.

Error Handling

Challenge: Managing API failures and invalid searches.

Solution: Added error messages and loading indicators.

Responsive Design

Challenge: Ensuring compatibility across devices.

Solution: Used CSS Flexbox and media queries.

10. LEARNING OUTCOMES.

Through this project, I gained practical experience in:

- React application development.
 - API integration and data fetching.
 - React Hooks (useState and useEffect).
 - Component-based architecture.
 - State management.
 - Conditional rendering techniques.
 - Responsive web design.
 - GitHub version control.
 - Debugging and troubleshooting React applications.
-

11. CONCLUSION

The API-Based COVID/Health Stats Tracker was successfully developed using React. The application provides real-time health statistics through API integration and allows users to search country-specific data efficiently. Conditional alerts help identify high-risk countries, making the platform useful for health awareness campaigns. The project achieved all intended objectives and enhanced understanding of React development, API handling, and dynamic web application design. The application can be further extended with advanced analytics, charts, and additional health-related features in future versions.
